

multi_maze_v2

Current control, simulation, and learning design

Goal: learn a reliable ball-maze policy from measured physics and model-predictive demonstrations, then adapt safely to real hardware.

Server demonstration generation

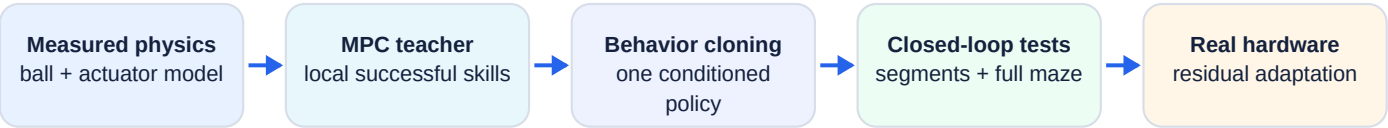
4,825 / 5,000 safely saved



96.5% complete

4 active CPU workers - 0 errors

System at a glance

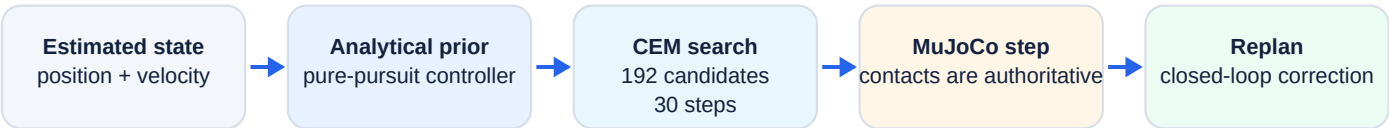


Evidence already established	Result
Representative MPC canary	75/75 goals; 0 falls; 0 timeouts
Geometry coverage	Authentic, mirrored-authentic, and procedural
Robustness coverage	Sensor noise and physics randomization up to 25%
Local episode policy	Completion first; 60 s maximum budget
Server execution	20 disjoint workers; resumable 25-episode shards

Design principle: known mechanics supply structure; learning is used for flexible route following, recovery, and the remaining sim-to-real gap.

1. Current MPC teacher design

The teacher replans at 20 Hz. It predicts candidate action sequences with an analytical rolling-ball and measured actuator surrogate, then executes only the first action in MuJoCo before planning again.



Teacher parameters

Element	Current setting	Reason
Planning horizon	30 control steps / 1.5 s	Anticipates momentum and braking
CEM population	192 candidates, 4 iterations, 24 elites	Broad search with bounded runtime
Analytical prior	Pure-pursuit velocity field	Known-safe feedback anchor
MPC residual	+/- 0.03 normalized action	Prevents model-error wall stalls
Execution plant	MuJoCo	Retains real contacts, walls, and holes
Local timeout	60 s	Allows robust completion under randomization

Why the teacher is hybrid

Known physics contributes	MuJoCo contributes	Learning later contributes
Gravity-driven ball acceleration; actuator dead time, lag, slew, and backlash; route geometry and stopping logic.	Wall and floor contacts; hole falls; exact maze geometry; authoritative next state.	One compact policy; recovery from unfamiliar states; residual compensation on the real rig.

Safety correction discovered by the canary: a 12% MPC residual degraded one mirrored sharp-right geometry. Reducing it to 3% and restoring the 60 s completion budget produced 75/75 representative successes.

2. Ball-on-plate mathematical model

This model lives inside the MPC prediction loop. For every candidate plate-command sequence, it predicts the ball trajectory cheaply; MuJoCo then executes the selected first action and supplies the authoritative contact-rich next state.



Rolling-sphere dynamics

x-axis motion

$$x'' = (5/7) g \sin(\beta) - c x'$$

y-axis motion

$$y'' = -(5/7) g \sin(\alpha) - c y'$$

Rolling gain: $(5/7) g = 7.007 \text{ m/s}^2$ per radian for small tilts

The 5/7 factor follows from a solid sphere rolling without slipping: part of gravity accelerates translation and part accelerates rotation. The fitted linear term c opposes velocity and approximates rolling resistance plus residual simulator damping.

Variables and sign convention

Symbol	Meaning	Project convention
alpha	Plate roll angle	Positive alpha accelerates the ball toward negative y
beta	Plate pitch angle	Positive beta accelerates the ball toward positive x
g	Gravity	9.81 m/s^2
c	Linear damping coefficient	ball.linear_damping from parameters.json
x', y'	Board-frame ball velocity	Estimated and latency-predicted state

Closed-form control-step integration

Update	Equation
Position	$p_{\text{next}} = p + v \, dt + 0.5 \, a \, dt^2$
Velocity	$v_{\text{next}} = v + a \, dt$

Implementation and limits

- sim/analytic_model.py contains the scalar equations, rollout, stopping-time, and switching-curve helpers.
- control/mpc_teacher.py applies the same equations to 192 candidate sequences in parallel.
- The predictor includes measured actuator dead time, first-order lag, slew limits, and backlash before the ball equations.

3. Balanced local-skill curriculum

The teacher demonstrates short maneuvers separately. Balanced sampling prevents abundant straight motion from dominating rare turns.

Maneuver	Geometries	Initial conditions	Episode target
Straight	50	20 per geometry	1,000
Gentle left	50	20 per geometry	1,000
Gentle right	50	20 per geometry	1,000
Sharp left	50	20 per geometry	1,000
Sharp right	50	20 per geometry	1,000
Total	250	5,000 specifications	5,000

Geometry sources

Source	Count	Purpose
Authentic	42	Local slices of the actual maze
Mirrored authentic	42	Valid reflected layouts with walls and holes reflected too
Procedural	166	Fills class imbalance and expands shape diversity

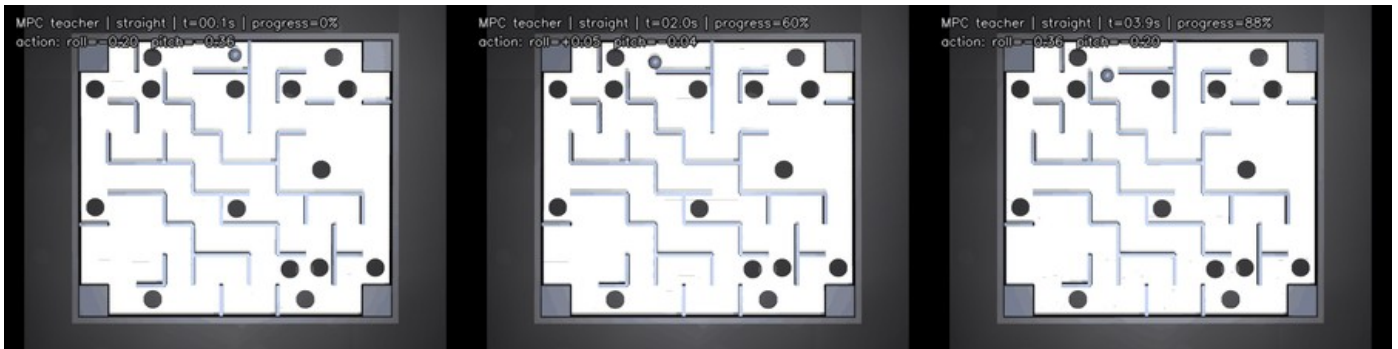
Initial-condition coverage

- Lateral offsets span both sides of the route centerline within local clearance.
- Forward and lateral velocities vary so the teacher demonstrates braking and recovery, not only rest starts.
- Physics randomization levels include nominal, 10%, and 25%, with sensor noise enabled.

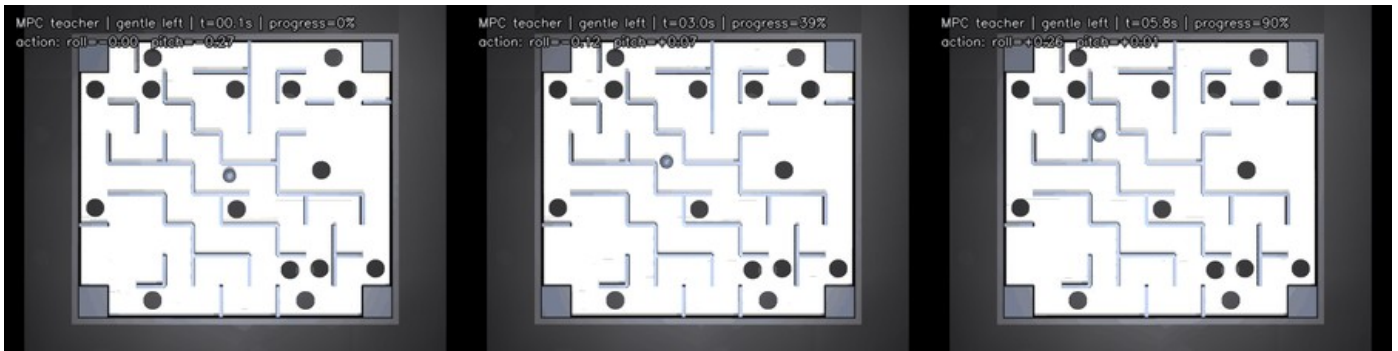
Representative teacher demonstrations

Each row shows the beginning, middle, and successful end of one authentic-maze local maneuver.

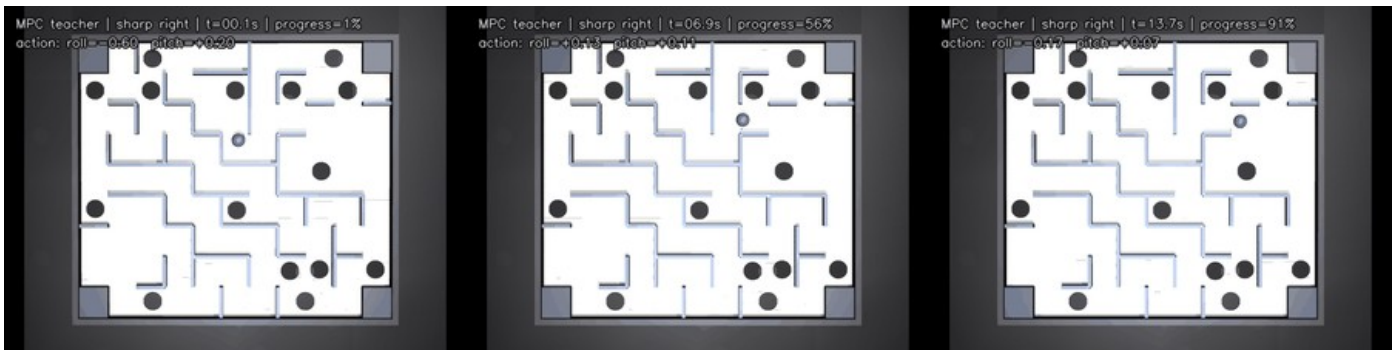
Straight - Goal in 3.85 s; mean cross-track 1.83 mm



Gentle left - Goal in 5.80 s; mean cross-track 2.53 mm

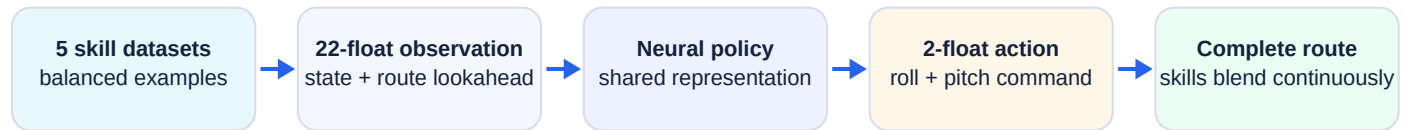


Sharp right - Goal in 13.70 s; mean cross-track 4.04 mm



4. One route-conditioned imitation policy

The short skills are not deployed as five separate controllers. Their successful transitions are pooled to train one policy that selects the appropriate behavior from the local route shape.



Policy contract

Observation block	Floats	Meaning
Ball position	2	Board-normalized x and y
Predicted ball velocity	2	Latency-compensated vx and vy
Board angles	2	Normalized roll and pitch
Action history	6	Three previous two-axis commands
Route lookahead	10	Five ball-relative points at 12 mm spacing
Total	22	State-based, route-conditioned observation

Behavior-cloning objective

For every saved teacher transition, minimize the squared difference between the policy action and the MPC action: $L = \text{mean } || \text{policy}(\text{observation}) - \text{teacher_action} ||^2$.

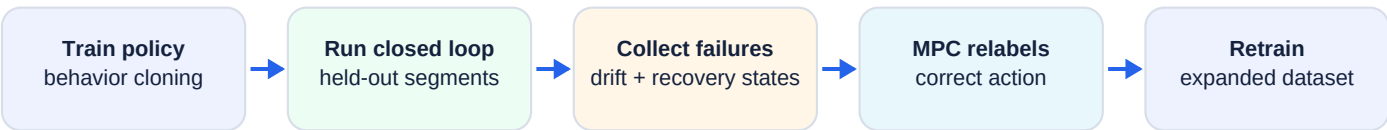
Training split and balancing

- Split by complete geometry, never by individual frame, so validation routes are genuinely unseen.
- Balance maneuver classes and emphasize recovery states instead of letting long straight trajectories dominate.
- Train on the server GPU, but judge models through closed-loop simulation rather than action loss alone.
- Save the best checkpoint by evaluation success, not merely the final epoch.

The route-conditioned policy does not explicitly choose 'turn left' or 'go straight.' Five relative lookahead points describe the local shape, and the shared network produces the matching action continuously.

5. Closed-loop evaluation and recovery

Low imitation loss is necessary but not sufficient. Small action errors can move the ball into states absent from the teacher data, so every candidate policy must be executed in simulation.



Evaluation ladder

Gate	Primary measures	Decision
Held-out local segments	Success by class; falls; completion; cross-track	Target $\geq 95\%$ success with near-zero falls
Unseen procedural routes	Generalization across radius, angle, and placement	Reject memorized geometry
Full nominal maze	Start-to-goal success and accumulated drift	Prove that local skills compose
Randomized simulation	Success under friction, latency, bias, and noise	Measure robustness before hardware
Hardware canary	Conservative speed, tilt, and emergency-stop behavior	Expand only after safe repeatability

Anti-collapse protections for later SAC fine-tuning

- Retain teacher demonstrations in replay rather than replacing them with weak on-policy behavior.
- Use an imitation regularizer so the actor cannot drift rapidly away from the successful teacher manifold.
- Use a small learning rate, periodic deterministic evaluation, and best-checkpoint protection.
- Stop or roll back when success falls, even if TensorBoard training reward rises.

Success is always an executed outcome. Route completion alone does not prove that the ball reached the goal, and training reward alone does not prove that the policy is stable.

6. Execution roadmap

Current server layout

Item	Current design
Remote project	/home/tn22833/multi_maze_v2
Teacher generation	20 disjoint CPU workers; 25 episodes per resumable shard
Hardware available	256 CPU cores; five RTX 6000 Ada GPUs with 49 GB each
Current status snapshot	4,825/5,000 saved; 4 workers active; 0 error logs
Next runtime	GPU-enabled PyTorch for behavior cloning and later SAC

Milestones from here

Stage	Work	Exit condition
A. Corpus closeout	Verify 5,000 episodes, unique IDs, readable shards, class/source balance	Integrity report passes
B. Behavior cloning	Train one route-conditioned policy on GPU	Validation loss stable; no train/validation leakage
C. Segment evaluation	Run held-out authentic, mirrored, and procedural routes	$\geq 95\%$ local success; near-zero falls
D. DAgger recovery	MPC relabels states reached by policy drift	Recovery failures materially reduced
E. Full-maze evaluation	Nominal and randomized start-to-goal runs	Reliable goal success, not completion only
F. Protected RL	Fine-tune with SAC plus demonstration retention	Beats imitation without checkpoint collapse
G. Hardware transfer	Conservative canary, system identification, residual adaptation	Safe repeatable real-maze runs

Final architecture



The intended result is not an MPC controller running permanently on the robot. MPC supplies high-quality behavior offline; the learned route-conditioned policy supplies fast online control; hardware data then corrects the remaining model mismatch.